

# “瑕宝阁”微瑕非遗产品交易平台—风险分析报告

团队编号：第 9 组

项目成员：2251310 杜天乐 2251916 罗诗雨 2252443 吴婉宁 2252444 赵思源

## 一、项目简述

“微瑕非遗产品交易平台”是一个融合电商交易、社区互动与非物质文化遗产推广于一体的复合型网站，旨在通过互联网手段，提升因微瑕而被传统渠道边缘化的非遗产品的市场流通效率和社会关注度。本平台聚焦“微瑕非遗”这一独特品类，打破传统审美与商业体系的限制，允许消费者以更亲民的价格购买到带有瑕疵但仍具审美与文化价值的非遗产品。

### 1. 核心目标

- 建立一个集商品交易、品牌认证、互动社区、积分支付等为一体的一站式平台
- 通过主题市集、品牌认证、积分激励等机制提升商家积极性与用户活跃度
- 形成以“微瑕”为主线的非遗文化传播生态圈

### 2. 系统用户视角支持

#### 2.1 买家视角

- 浏览、筛选与搜索非遗商品
- 收藏、下单、支付与售后服务
- 社区发帖、点赞评论、举报维权
- 钱包充值与积分使用

#### 2.2 商家视角

- 注册认证与品牌审核
- 商品信息上架/下架与编辑
- 参与市集与定向折扣
- 查看订单、营收统计、售后管理
- 查看店铺评价与用户互动

#### 2.3 管理员视角

- 平台用户权限管理
- 审核商品与商家认证
- 发布与管理“主题市集”
- 处理举报与维权仲裁
- 监管社区内容与用户行为

### 3. 技术架构与实现细节

平台整体采用 **前后端分离架构**，具体技术栈包括：

| 层级    | 技术选型                           | 功能说明              |
|-------|--------------------------------|-------------------|
| 前端    | Vue.js + Element UI            | 实现响应式界面与用户交互      |
| 后端    | ASP.NET Core MVC               | 提供 Web API 支撑业务逻辑 |
| 数据层   | Oracle + Entity Framework Core | 支持强类型 ORM 映射和多表联查 |
| 中间件   | Nginx + Cookie 登录鉴权            | 统一网关、负载均衡与会话管理    |
| 工具集成  | ApiFox + GitHub + 腾讯会议         | 接口管理、版本控制与远程协作    |
| 第三方服务 | 支付宝支付 SDK + 邮箱验证码服务            | 实现安全高效的支付与认证机制    |
| 标识生成  | Yitter.IdGenerator 雪花算法        | 保证分布式系统中的唯一 ID 分配 |

### 4. 数据与功能模块概览

平台功能模块共包含 **30+个详细用例**，涵盖了从“用户注册登录”到“支付确认收货”再到“社区互动与申诉处理”的完整交易流程，并配套有：

- 钱包模块、订单模块、积分系统
- 品类管理、收藏夹、论坛系统
- 商品图文上传、多级分类与搜索
- 店铺评价、可视化营收统计等多样业务组件。

数据库设计具备 **完善的主外键结构与冗余优化策略**，表结构超 30 张，支持高并发、并行访问及复杂事务处理。

### 5. 测试重要性说明

由于该系统为多角色多模块复合系统，且涉及资金支付、用户评价和社会性话题传播，其稳定性与安全性对平台口碑与后续推广具有决定性意义。特别是在以下方面必须重点测试与评估：

- 交易正确性（订单金额、支付、积分抵扣）
- 权限安全性（角色越权、数据访问隔离）
- 系统性能（并发下市集活动、热销商品高频访问）
- 可用性与容错性（支付中断、社区举报、申诉流程等异常场景）

风险分析将以此为出发点，逐一识别潜在问题，提供预警与预案，协助系统测试工作的全面展开。

## 二、系统架构相关技术风险

本平台采用 ASP.NET Core MVC 框架 + Entity Framework Core ORM + Oracle 数据库的技术栈，整体技术选型具备高可维护性和稳定性，但在开发与部署过程中，**底层架构与依赖工具**可能成为潜在风险源。

| 风险名称        | 描述                                                                                                 | 影响范围           | 风险等级 | 预防策略                                                               |
|-------------|----------------------------------------------------------------------------------------------------|----------------|------|--------------------------------------------------------------------|
| ORM 映射出错    | Entity Framework Core 使用 Code First 映射方式时，若类字段未正确标注、缺少主键或映射异常，可能导致数据库字段无法正确绑定或读取，进而引发业务逻辑错误或运行时崩溃。 | 数据访问层故障、业务逻辑出错 | 高    | 编写字段与表结构对应的单元测试；使用 Scaffold 工具自动生成模型，避免手动误差；在开发阶段就启用迁移与差异检查。       |
| 雪花 ID 生成冲突  | 使用 Yitter.IdGenerator 实现全局唯一 ID 分配时，如未设置机器码或配置冲突，可能导致重复 ID，进而引起主键冲突、数据插入失败甚至数据污染。                  | 主键冲突，数据覆盖      | 高    | 每个节点明确配置不同机器码；系统启动前进行分布式 ID 生成器单测；日志记录分布式 ID 分配过程。                 |
| Cookie 鉴权失效 | 若客户端 Cookie 被恶意篡改、浏览器禁用 Cookie，或服务端未设置有效时长、路径、作用域等，可能导致用户权限失效、跳转异常甚至越权访问。                          | 用户认证失败、出现越权访问  | 中    | 引入身份签名机制 + CSRF Token 防伪；所有敏感接口加入权限判断逻辑；设置 Cookie Scope 限定域名与过期策略。 |

### 三、功能模块详细风险分析

#### 3.1 用户注册与登录模块

该模块是用户接触系统的第一入口，其安全性与可用性直接影响平台用户留存率。尤其涉及邮箱验证码、密码存储、角色识别等环节，若设计不当，可能引发信息泄露、权限错配与系统漏洞。

| 风险项      | 描述                                  | 潜在问题             |
|----------|-------------------------------------|------------------|
| 邮箱验证不稳定  | 依赖第三方邮件服务发送验证码，存在失败、延迟、邮箱误判为垃圾邮件等问题 | 用户收不到验证码，注册/找回失败 |
| 身份判定错误   | 注册或登录时后台未正确识别角色字段（如买家/商家/管理员）       | 权限错配，可能获取非法访问    |
| 密码明文存储风险 | 如果后台未加密处理密码，或未正确使用加盐算法              | 密码泄露后整个平台安全体系崩溃  |

应对措施：

- 后端采用 `BCrypt` 等带加盐机制加密密码，防止彩虹表攻击
- 邮箱验证码应设置发送频率限制与有效期，支持异常重发机制
- 注册与登录流程中加入字段校验与单元测试，确保身份分类准确性

### 3.2 商品发布与订单模块

该模块为交易主流程，是平台收益的核心来源。涉及商品库存状态、积分抵扣、订单生成、优惠逻辑等多个精细操作，需特别注意逻辑完整性与并发一致性。

| 风险项       | 描述                           | 影响              |
|-----------|------------------------------|-----------------|
| 商品状态异常    | 商品标记为“在售”但已被其他用户购买，或售出后未及时更新 | 多用户重复下单，影响信誉    |
| 订单金额计算误差  | 积分抵扣、满减、优惠券等叠加逻辑未统一精度处理      | 实际支付金额错误，商家收入不准 |
| 支付过程未验证库存 | 生成订单时未及时检查库存，导致支付后无法发货       | 出现“超卖”，需退款，影响体验 |

应对措施：

- 商品下单时强制校验库存状态，并使用分布式乐观锁
- 所有金额字段统一使用 Decimal(18,2) 且不得参与前端浮点计算
- 后端设置冗余库存状态字段，支持商品回滚或自动取消订单

### 3.3 钱包与支付模块

由于该模块涉及用户资金与第三方支付平台交互，是全平台中最需保障事务一致性和幂等性的部分，任何处理失误都可能带来直接经济损失。

| 风险项      | 描述                         | 风险等级 | 风险后果         |
|----------|----------------------------|------|--------------|
| 钱包余额异常   | 并发支付中如不使用事务，可能导致余额重复扣除或未扣除 | 高    | 用户资金损失、纠纷频发  |
| 支付宝接口失败  | 支付过程中支付宝回调丢失或超时未处理         | 高    | 用户付款成功但系统未记账 |
| 积分抵扣机制绕过 | 用户通过刷新或篡改请求参数绕过抵扣限制        | 中    | 平台损失，破坏信用体系  |

应对措施：

- 支付与余额变动操作必须封装在事务内
- 设计回调重放机制，使用订单状态 + 校验码控制幂等处理
- 使用服务端校验积分抵扣额度，记录每笔积分扣除流水

### 3.4 论坛与互动模块

社区模块旨在提升平台用户粘性，但由于其开放性，容易成为违规内容传播、恶意行为发生的场所，需重点防范舆情与攻击行为。

| 风险项        | 描述                   | 风险等级 | 应对措施                     |
|------------|----------------------|------|--------------------------|
| 恶意内容上传     | 发帖/评论中夹带敏感词、黄暴图或虚假链接 | 高    | 整合图文内容识别 API，设定人工复审通道    |
| 评论刷赞刷评论    | 利用自动脚本刷数据干扰热门排序      | 中    | 设置 IP 频率限制、加入验证码机制       |
| 消息通知未读处理错误 | 多设备消息同步未正确标记读取状态     | 中    | 后端优化消息状态结构，采用“已读标识位”精确记录 |

### 3.5 社区举报与审核模块

该模块处理用户申诉与违规内容举报，对平台公信力起到关键作用，需保证处理及时性与准确性。

| 风险项     | 描述             | 后果                | 应对措施                   |
|---------|----------------|-------------------|------------------------|
| 举报处理不及时 | 管理员未及时处理用户举报内容 | 平台失去中立形象，用户不再积极参与 | 构建自动推送审核池，设置 24h SLA   |
| 举报冤假错判  | 非违规内容被误删或禁言    | 导致用户不满并流失         | 构建“申诉-复核”流程，管理员操作须审计记录 |

## 四、测试环境与部署风险

测试环境的不一致与部署流程的缺陷是导致线上问题的主要原因之一。应保障测试环境、预发布环境与生产环境高度一致，并在版本上线流程中引入回滚与数据备份能力。

| 风险项    | 描述                 | 风险等级 | 解决策略                         |
|--------|--------------------|------|------------------------------|
| 测试数据污染 | 共享数据库环境导致用户数据被覆盖   | 中    | 每次测试用例执行前后清理临时数据             |
| 环境差异   | 配置文件、数据库版本不一致      | 高    | 使用 Docker 或 CI/CD 管理全环境部署一致性 |
| 缺少回滚机制 | 新版本发布后发现致命错误无法快速撤回 | 高    | 配置版本切换机制与数据库自动快照备份           |

## 五、第三方依赖风险

本系统依赖多个外部 SDK 与服务接口，包括支付、邮件、登录鉴权等，这些第三方服务的稳定性与安全性直接影响系统可用性。

| 组件         | 风险                      | 建议                                |
|------------|-------------------------|-----------------------------------|
| 支付宝 SDK    | 新版本可能更改回调参数、签名算法等，造成不兼容 | 建立 SDK 升级跟踪机制，接口统一封装避免硬编码         |
| 邮件服务       | 邮件可能因封锁、地址黑名单等原因被退信     | 支持备用服务商切换机制，支持发送失败重试逻辑            |
| Oracle 数据库 | 商业授权费用高、与开源组件兼容性不足      | 审核授权条款，评估未来是否迁移至 PostgreSQL 等开源方案 |

## 六、未来扩展风险

平台上线初期可能功能较稳定，但随着用户量、城市扩张、交易增长，部分架构设计若未预留可扩展性将成为未来瓶颈。

| 风险项   | 描述                  | 对策建议                       |
|-------|---------------------|----------------------------|
| 用户量剧增 | 短时间内大量用户注册、并发交易请求   | 优化负载均衡与缓存策略，数据库读写分离        |
| 多语种支持 | 若开拓海外市场，界面与数据需支持多语言 | 使用 i18n 国际化框架；后端返回结构支持语言参数 |
| 法律合规  | 不同地区电商政策（如退货、税收）差异  | 模块化规则配置，支持“按地域”动态策略切换      |

## 七、风险矩阵与测试优先级总览

为进一步量化和系统化本平台在各功能模块中的潜在风险，结合前述各类风险源，从 **质量风险、技术风险、业务风险** 三个维度出发，构建统一的 **风险矩阵表格体系**。每一类风险均按模块归类编号，并依照“技术风险 × 业务风险”计算风险优先级，划分测试覆盖程度，便于测试团队对高风险模块优先开展全面测试与监控。

测试覆盖程度说明：

- 深入测试：**需设计全路径用例、异常处理与并发场景验证
- 常规测试：**涵盖基本功能流程、主路径用例验证
- 抽查测试：**仅对低风险模块进行接口通畅与边界验证

表 1 系统架构相关技术风险

| 编号 | 质量风险 | 风险项         | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|-------------|------|------|-------|--------|
| A1 | 功能性  | ORM 映射出错    | 5    | 4    | 20    | 深入测试   |
| A2 | 稳定性  | 雪花 ID 冲突    | 5    | 5    | 25    | 深入测试   |
| A3 | 安全性  | Cookie 鉴权失效 | 4    | 3    | 12    | 常规测试   |

表 2 用户注册与登录模块

| 编号 | 质量风险 | 风险项      | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|----------|------|------|-------|--------|
| B1 | 安全性  | 邮箱验证码不稳定 | 4    | 4    | 16    | 常规测试   |
| B2 | 安全性  | 身份判定错误   | 4    | 4    | 16    | 常规测试   |
| B3 | 安全性  | 密码明文存储风险 | 5    | 5    | 25    | 深入测试   |

表 3 商品发布与订单模块

| 编号 | 质量风险 | 风险项        | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|------------|------|------|-------|--------|
| C1 | 稳定性  | 商品状态异常（超卖） | 4    | 4    | 16    | 常规测试   |
| C2 | 功能性  | 订单金额计算误差   | 5    | 4    | 20    | 深入测试   |
| C3 | 功能性  | 支付未验证库存    | 4    | 5    | 20    | 深入测试   |

表 4 钱包与支付模块

| 编号 | 质量风险 | 风险项        | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|------------|------|------|-------|--------|
| D1 | 安全性  | 钱包余额并发扣款异常 | 5    | 5    | 25    | 深入测试   |
| D2 | 稳定性  | 支付宝回调丢失或超时 | 5    | 4    | 20    | 深入测试   |
| D3 | 安全性  | 积分抵扣机制绕过   | 3    | 4    | 12    | 常规测试   |

表 5 论坛与互动模块

| 编号 | 质量风险 | 风险项           | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|---------------|------|------|-------|--------|
| E1 | 安全性  | 恶意内容上传（黄暴/钓鱼） | 5    | 5    | 25    | 深入测试   |
| E2 | 可用性  | 评论刷赞刷评论       | 3    | 3    | 9     | 抽查测试   |
| E3 | 稳定性  | 消息未读状态同步错误    | 3    | 3    | 9     | 抽查测试   |

表 6 社区举报与审核模块

| 编号 | 质量风险 | 风险项     | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|---------|------|------|-------|--------|
| F1 | 稳定性  | 举报处理不及时 | 3    | 4    | 12    | 常规测试   |
| F2 | 可用性  | 举报冤假错判  | 3    | 3    | 9     | 抽查测试   |

表 7 测试环境与部署风险

| 编号 | 质量风险 | 风险项         | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|-------------|------|------|-------|--------|
| G1 | 稳定性  | 测试数据污染      | 3    | 3    | 9     | 常规测试   |
| G2 | 稳定性  | 环境差异（配置不一致） | 4    | 4    | 16    | 常规测试   |
| G3 | 稳定性  | 缺少回滚机制      | 5    | 5    | 25    | 深入测试   |

表 8 第三方依赖风险

| 编号 | 质量风险   | 风险项            | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|--------|----------------|------|------|-------|--------|
| H1 | 可维护性   | 支付宝 SDK 不兼容    | 4    | 4    | 16    | 常规测试   |
| H2 | 稳定性    | 邮件服务退信或延迟      | 3    | 4    | 12    | 常规测试   |
| H3 | 成本与合规性 | Oracle 授权及兼容问题 | 3    | 5    | 15    | 常规测试   |

表 9 未来扩展风险

| 编号 | 质量风险 | 风险项         | 技术风险 | 业务风险 | 风险优先级 | 测试覆盖程度 |
|----|------|-------------|------|------|-------|--------|
| I1 | 性能   | 用户量剧增       | 4    | 5    | 20    | 深入测试   |
| I2 | 可用性  | 缺乏多语种支持     | 3    | 3    | 9     | 抽查测试   |
| I3 | 合规性  | 不同地区电商法规不一致 | 3    | 5    | 15    | 常规测试   |

通过系统性地识别“微瑕非遗产品交易平台”在架构、功能、支付、社区、部署等模块的潜在风险，结合测试优先级与覆盖策略，为后续测试计划与系统上线提供了明确的方向与保障。随着项目推进，风险矩阵也将动态调整，以确保平台上线后具备高稳定性、安全性与用户满意度。

## 八、风险控制流程建议

为确保测试工作与系统交付的高质量推进，本项目建立了一套全流程的风险控制机制，覆盖测试前、中、后各阶段，保障问题能够提前识别、实时监控、快速响应。

在 **测试前阶段**，项目团队将构建《风险识别表》，逐一识别平台各模块可能存在的关键风险点，并评估其影响范围、发生概率与风险等级，形成风险优先级矩阵。基于风险等级，确立测试策略与优先级顺序。同时，制定完整的回归测试计划，明确每轮版本迭代的测试覆盖范围与测试周期，确保测试资源有效投入。



在 **测试执行阶段**，将采用“自动化 + 手动测试”相结合的策略。对于接口层面的测试，使用 Postman 或 ApiFox 执行请求压测与功能验证；同时引入 Mock 技术模拟第三方支付异常、网络不稳定等场景，评估系统的容错与降级能力。此外，围绕“用户注册 → 商品浏览 → 下单支付 → 评论反馈”的关键用户路径，制定端到端测试用例，确保核心交易闭环的稳定与一致性。

在 **部署前阶段**，系统需在预发布环境中完成全量回归测试，并同步比对数据结构、配置文件与数据库状态，防止环境不一致问题。上线版本须通过严格审核并锁定版本号，确保无临时变更影响稳定性。同时将采用灰度发布策略，按用户或城市分批上线，并实时监控关键指标变化，降低风险影响面。

在 **部署后阶段**，平台将启动实时监控体系，使用 Prometheus + Grafana 采集和展示接口响应时间、数据库负载、异常日志等系统指标，便于快速定位与响应问题。同时，预设应急回滚机制，保留上一个稳定版本的数据库备份与应用快照，以便故障恢复。在用户层面，设立问题反馈通道，安排专人值守响应用户反馈，确保问题发现即处理、处理即闭环。